DDM501 – AI in DevOps, DataOps, MLOps

Individual Assignment 2

ML Pipeline Design & MLOps Analysis

Perishable-Food Demand Forecasting
and Replenishment Assistant

Executable prototype, experiment evidence, and production workflow

---

Student: Tran Viet Hoa
Student ID: 25MS23263

Course: DDM501
Assignment weight: 5%
Date: October 4, 2026

All experiment results were produced by the companion submission ZIP.
The public dataset is a structural proxy and not FreshMart production data.

Contents

Executive Summary **1**

1 Continuity from Assignment 1 **1**
1.1 Problem statement and prior design 1

2 Pipeline Design (25%) **2**
2.1 End-to-end architecture and data flow 2
2.2 Stage specifications and quality gates 2
2.3 Design rationale 3

3 Experiment Tracking and Metrics Analysis (25%) **4**
3.1 Dataset, protocol, and leakage controls 4
3.2 Variables, baselines, and experiment matrix 4
3.3 Metrics strategy and business alignment 5
3.4 Tracking, artifacts, and model registration 6
3.5 Validation results and analysis 7
3.6 Untouched test-quarter evidence 8

4 Workflow Orchestration Design (20%) **8**
4.1 DAG architecture and dependencies 9
4.2 Airflow definition and operational controls 10

5 Code Quality and Documentation (20%) **12**
5.1 Standards 12
5.2 Configuration and environment variables 13

6 Reproducibility and Versioning Strategy (10%) **14**
6.1 Code, data, configuration, and model versions 14
6.2 Determinism and containerization 15

7 Limitations, Risks, and Next Decision **16**

8 Conclusion **16**

A Rubric-to-Artifact Traceability **17**

Executive Summary

This report continues the production ML system proposed in Assignment 1: a nightly assistant that forecasts demand for each store–SKU, converts the forecast into a constrained replenishment recommendation, and keeps the store manager in control [1]. The architecture is refined into two explicit Airflow workflows: a nightly inference DAG with a 03:00 service-level objective and a weekly, drift-triggerable training DAG. Both reuse one point-in-time feature implementation; MLflow provides experiment lineage and the registry hand-off; data and configuration are checksum/version controlled; failures degrade to a stale forecast or a seven-day seasonal-naive baseline.

Because the hypothetical FreshMart data is unavailable, an executable prototype was evaluated on the public Store Item Demand Forecasting data: 913,000 daily observations, 10 stores, 50 items, and five years [2]. A fixed SHA-256 checksum, causal lag features, and chronological partitions prevent silent data changes and time leakage. Twelve configurations were declared and run initially on 456,000 training rows and 46,000 validation rows. E12 was added after review as a clean `log1p` preprocessing ablation; it did not change the selected candidate. The selected histogram gradient-boosted regressor using Poisson loss obtained 9.73% validation WAPE and −0.35% aggregate bias. After selection, it was retrained on train plus validation and evaluated once on the untouched 46,000-row test quarter: WAPE was 10.94%, versus 16.60% for seasonal naive, a 5.66 percentage-point or 34.1% relative improvement. Its asymmetric inventory-cost proxy also fell from 0.237 to 0.165 per actual unit.

These are prototype accuracy results, not FreshMart business outcomes. The proxy lacks inventory, promotions, weather, shelf life, prices, stockout censoring, and three-day horizons; its fill metric is not on-shelf availability and its overage proxy is not waste. Therefore E10 is registered only as a *candidate*. Promotion to champion requires rolling-origin evaluation on FreshMart data, calibrated quantiles, segment gates, shadow operation, and the matched-store pilot defined in Assignment 1.

1 Continuity from Assignment 1

1.1 Problem statement and prior design

FreshMart Vietnam is a hypothetical 350-store convenience chain with about 220 fresh SKUs per store and one-to-five-day shelf lives. Managers currently order manually using a seven-day moving average. Assignment 1 set the production objective: predict next-day through day-three unit demand, reduce forecast WAPE from 38% to at most 27%, reduce fresh-food waste from 9% to at most 6% of COGS, and raise on-shelf availability from 89% to at least 95% [1]. The decision is asymmetric: under-ordering loses margin and customer trust, while over-ordering creates spoilage.

The earlier design selected one global LightGBM model, split-conformal uncertainty, and a separate newsvendor order engine. It was batch-first because ordering is daily, human-in-the-loop because local events are incompletely observed, and designed for graceful degradation. This report retains those principles while making the following refinements:

- training and inference are separate DAGs connected only by an immutable registry version;
- every target-derived feature is shifted before rolling aggregation, and all joins are point-in-time correct;
- validation, promotion, alerting, and fallback behavior are executable workflow stages rather than narrative controls;
- experiment selection uses an untouched final quarter and explicit WAPE, bias, and baseline-improvement gates;
- the local prototype uses scikit-learn histogram gradient boosting as a dependency-light GBDT proxy. The production recommendation remains LightGBM after benchmarking on FreshMart-scale data [3].

2 Pipeline Design (25%)

2.1 End-to-end architecture and data flow

Figure 1 shows the detailed production pipeline. Solid named arrows identify the transformation at every hand-off; dashed arrows show cross-cutting control and feedback. Separate boxes now make preprocessing, evaluation, and deployment explicit. Raw inputs are immutable. Failed partitions are quarantined rather than partially published. A single causal feature definition serves training and inference, avoiding training–serving skew. Forecasting is kept separate from the deterministic replenishment policy, so pack size, minimum order quantity (MOQ), lead time, shelf life, inventory, and cut-off constraints can be unit-tested without retraining the model.

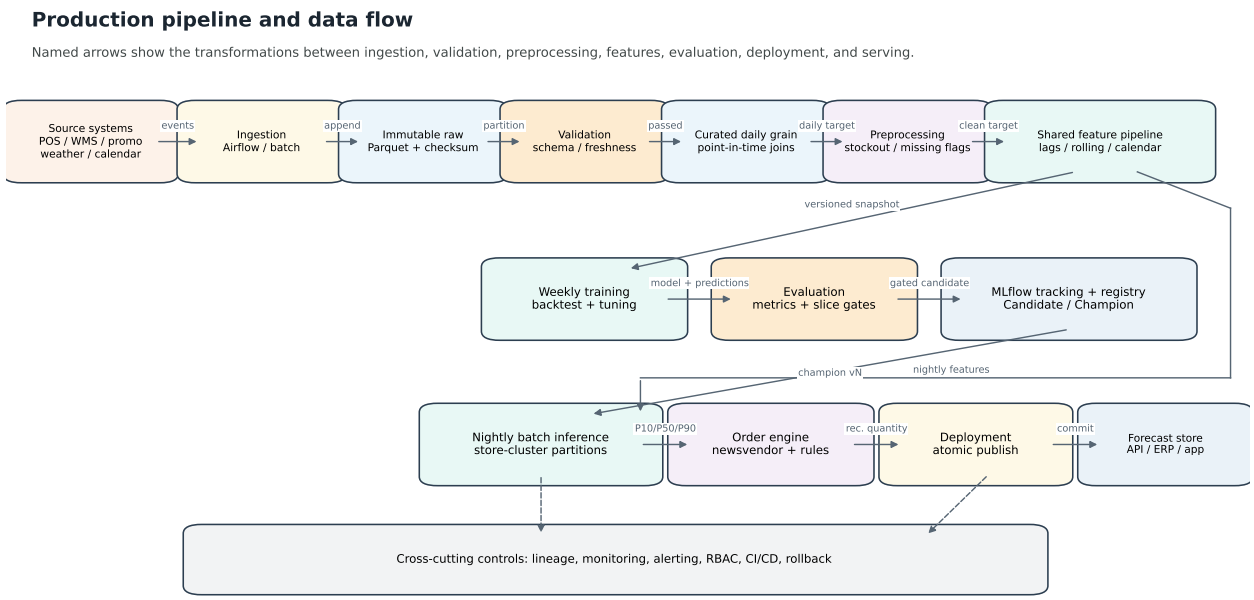


Figure 1: Production pipeline architecture and major data transformations.

2.2 Stage specifications and quality gates

Table 1: Pipeline stages, contracts, and fail-closed gates.

Stage	Purpose	Input → output	Key operations	Quality gate
Ingest	Preserve source facts and lineage	POS, WMS, masters, price/promo, weather/calendar → date-partitioned Parquet	Idempotent extract; schema metadata; source timestamp; checksum	At least 98% stores by 00:30; expected partition and checksum
Validate	Stop corrupt or late data	Raw partitions → pass report plus quarantine list	Type/range/schema, uniqueness, freshness, completeness, referential integrity	No negative net sales except tagged returns; duplicate fraction $< 10^{-4}$
Curate	Establish daily analytical grain	Valid events → store–SKU–day tables	Deduplicate; returns policy; aggregate; point-in-time master joins; stockout flag	Row balance to source; unique composite key; no future timestamps

Stage	Purpose	Input → output	Key operations	Quality gate
Preprocess	Produce model-ready targets	Curated tables → censored-demand-aware target	Missing flags; stockout down-weight/imputation; category fallback keys	Imputation rate and censored-day share within learned control limits
Feature engineering	Reuse causal predictors	Target history and known-ahead covariates → versioned feature snapshot	Lags 1/7/14/21/28; shifted rolling windows; calendar; promo; forecast weather	No target leakage; training/offline parity; nulls only during declared warm-up
Train and tune	Fit a global model	Versioned snapshot + YAML configuration → model artifact	Rolling-origin folds; fixed seed; parameter matrix; segment metrics	Challenger beats both baselines; $\lvert\text{bias}\rvert \leq 3\%$; no critical segment regression
Evaluate	Measure fitness and risk	Predictions + actuals → signed evaluation report	WAPE, bias, RMSE, sMAPE, interval coverage, slice analysis, runtime	WAPE $\leq 27\%$; P90 coverage 88–92%; reproducibility check
Register	Control model handoff	Model, signature, metrics, data/code versions → immutable candidate version	Log artifacts; attach model card; approval; alias transition	Complete lineage; tests pass; no direct overwrite of champion
Batch inference	Forecast all active series	Champion + nightly features → mean and P10/P50/P90 forecasts	Partition by store cluster; schema/signature validation; non-negative output	Coverage 100% active pairs or explicit fallback; finish before 02:30
Order engine	Convert demand to action	Quantiles + inventory + rules → order quantity	Newsvendor quantile; net on-hand/in-transit; round to pack; MOQ/-caps	Deterministic constraint tests; anomaly cap; kill switch
Deploy/publish	Serve manager and ERP	Versioned recommendations → atomic forecast-store commit and export	Blue/green table; REST/API cache; SFTP/ERP export; lineage fields	Row count, referential integrity, read-after-write; ready by 03:00
Monitor	Close the loop	Logs, features, predictions, actuals, overrides → dashboards/alerts	SLA/error rate; PSI/drift; delayed WAPE/bias; waste/OSA; override FVA	Page on data/SLA failure; retrain on sustained drift; rollback on regression

2.3 Design rationale

Modularity and reuse. Each stage consumes and produces a versioned contract. Extractors are source-specific, while validation, feature calculation, metrics, tracking, and registry interfaces are reusable. The order engine depends on a forecast schema rather than a model class. Consequently LightGBM can later replace the prototype GBDT without changing downstream integrations. This favors the accuracy/complexity profile observed in large retail benchmarks [4] while

containing the hidden technical debt created by unnecessary pipeline variants [10]. The executable repository mirrors this separation through `src/data.py`, `src/metrics.py`, `src/experiment.py`, and `src/tracking.py`.

Error handling and recovery. Writes are idempotent and land in run-specific staging paths before an atomic publish. Transient tasks retry three times with exponential backoff; validation failures are not retried blindly because bad data will remain bad. They quarantine the partition and alert the owner. Recovery tiers are: (1) use the last successful forecast if it is under 30 hours old; (2) recompute seven-day seasonal naive in SQL; and (3) return control to the current manual process. A champion alias can be rolled back to an immutable previous version without rebuilding an image.

Scalability. Today there are about 77,000 store–SKU series and roughly 56 million rows in a 24-month window. Warehouse tables are partitioned by date and clustered by store/SKU. Batch inference partitions by store cluster; workers are stateless, so Kubernetes can add partitions without changing business logic. At projected 5× growth, feature computation can move from warehouse SQL/Pandas to Spark while preserving the same feature contract. Model training remains one global model; per-series models would create tens of thousands of deployment objects.

3 Experiment Tracking and Metrics Analysis (25%)

3.1 Dataset, protocol, and leakage controls

The public Kaggle Store Item Demand Forecasting data contains daily sales for 50 items in 10 stores from 2013-01-01 through 2017-12-31 [2]. The downloaded file contains 913,000 rows and 500 complete series. The experiment runner verifies the exact SHA-256 digest

`038f25690a65149c94f86ddd3deceda20c037a5cfd754cafdfc539a72992f2ed`.

The data-quality run found zero duplicate keys, missing required fields, negative targets, or within-series daily gaps. After the 28-day warm-up, 899,000 rows remained. The fixed chronological split was:

Table 2: Chronological experimental partitions. The test quarter was opened only after selection.

Partition	Rows	Series	Dates
Train	456,000	500	2015-01-01 to 2017-06-30
Validation	46,000	500	2017-07-01 to 2017-09-30
Test	46,000	500	2017-10-01 to 2017-12-31

Random splitting would allow future seasonal states from the same series into training, so it was rejected. All rolling values use `shift(1)` before aggregation. The experiment is one-day-ahead because the proxy has no known-ahead promo/weather inputs. The production design still creates $h \in \{1, 2, 3\}$ examples with horizon as a feature.

The raw history starts in 2013, but model fitting begins on 2015-01-01. The first 351,000 post-warm-up observations (2013-01-29 through 2014-12-31) are intentionally excluded to use a recent multi-year window: 30 months in this public-data prototype versus the 24-month window used for production sizing in Section 2.3. This bounds training cost and reduces the influence of obsolete levels. The older rows remain available for alternate-window sensitivity tests. Calendar trend is anchored to the configured fixed date 2013-01-01, rather than to the first row of whichever frame is passed, so training and inference compute identical values.

3.2 Variables, baselines, and experiment matrix

The controlled variables were algorithm, objective, feature group, target preprocessing, learning rate, maximum iterations, leaf count, minimum leaf size, and L2 regularization. E00 is the required

seasonal-naive baseline (same weekday last week), E01 is the current-process analogue (seven-day moving average), and E02 is a regularized linear reference. E03–E12 are histogram GBDT variants. Three feature groups isolate the value of added information:

- **minimal**: store, item, weekday, month, lag 1, and lag 7;
- **core**: minimal plus lags 14/28, shifted rolling mean/standard deviation, and four-week same-weekday mean;
- **extended**: core plus lag 21, trend, day-of-year, and cyclic calendar encodings.

Table 3: Matrix of 13 executed configurations. E00–E11 were declared initially; E12 is the post-review preprocessing ablation. LR is learning rate; leaves is maximum leaf nodes; iter is boosting iterations.

ID	Algorithm/objective	Features	LR	Leaves	Iter	L2	Min leaf
E00	Seasonal naive	baseline	–	–	–	–	–
E01	Moving average	baseline	–	–	–	–	–
E02	Ridge ($\alpha = 10$)	core	–	–	–	10	–
E03	HGB squared	minimal	.08	31	120	0	40
E04	HGB squared	core	.08	15	140	0	40
E05	HGB squared	core	.06	31	180	0	40
E06	HGB squared	core	.05	63	200	0	30
E07	HGB squared	extended	.05	31	220	1	40
E08	HGB squared	extended	.04	63	260	1	30
E09	HGB Poisson	extended	.05	31	220	1	40
E10	HGB Poisson	extended	.04	63	280	2	30
E11	HGB squared	extended	.03	31	300	5	80
E12	HGB squared + `log1p`	extended	.05	31	220	1	40

E12 was added after review to make preprocessing explicit; it did not affect selection. E07 and E12 are a clean preprocessing comparison: all features and hyperparameters are identical, while E12 trains on $\log(1 + y)$ and inverts predictions with `expm1`. Every other configuration keeps the target in raw non-negative units. Likewise, E07 versus E09 isolates squared versus Poisson loss; E08 versus E10 does not, because iterations and L2 also change.

3.3 Metrics strategy and business alignment

For observations y_i and non-negative predictions $\hat{y}_i$, the primary metric and aggregate bias are

$$\text{WAPE} = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i y_i}, \qquad \text{Bias} = \frac{\sum_i (\hat{y}_i - y_i)}{\sum_i y_i}. \tag{1}$$

WAPE is stable when low-volume series include zeros, unlike MAPE [5]. Bias is a hard gate because a low-WAPE model can still systematically under-order. RMSE emphasizes peak errors; MAE is interpretable in units; sMAPE supports comparison with the Kaggle convention. Metrics must also be sliced by horizon, category, store cluster, weekday, promotion, holiday, and cold-start status in production.

To expose asymmetric decisions, the prototype reports underage $U_i = \max(y_i - \hat{y}_i, 0)$, overage $O_i = \max(\hat{y}_i - y_i, 0)$, and

$$C_{\text{proxy}} = \frac{2 \sum_i U_i + \sum_i O_i}{\sum_i y_i}. \tag{2}$$

The 2:1 weight is a declared planning assumption, not a measured VND cost. The associated demand-fill proxy is $\sum_i \min(y_i, \hat{y}_i) / \sum_i y_i$. In production, forecast metrics enable business goals but do not replace them: the 8-week controlled pilot must measure waste/COGS, on-shelf availability, gross margin, manager time, adoption, and forecast value added.

3.4 Tracking, artifacts, and model registration

Every run records the full YAML parameters, seed 501, dataset digest, Git commit SHA, feature set, target transform, validation metrics, runtime, and environment-independent identifiers. Filesystem JSON is always written for inspectability; the same record is mirrored to MLflow when the optional service is enabled. Predictions, plots, the selected serialized model, data-quality summary, results CSV, test metrics, and candidate manifest are logged artifacts.

Listing 1: MLflow experiment, parameter, metric, and lineage-tag logging.

```
    mlflow.set_experiment(experiment_name)
    with mlflow.start_run(run_name=run_name):
        mlflow.log_params({key: str(value) for key, value in params.items()})
        mlflow.log_metrics(
            {key: float(value) for key, value in metrics.items() if isinstance(
                value, (int, float))}
        )
        mlflow.set_tags(
            {
                "dataset_role": "public-proxy",
                "task": "one-day-demand",
                "git_commit": git_commit_sha(),
            }
        )
```

Registration is executable rather than narrative-only: evidence files are attached to the run, `mlflow.sklearn.log_model` creates a new registry version, and the client assigns the `candidate` alias to that returned version.

Listing 2: Artifact logging, numeric version capture, release-ID tag, and candidate alias.

```
    for artifact in artifacts:
        mlflow.log_artifact(str(artifact), artifact_path="evidence")
    model_info = mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path="model",
        registered_model_name=registered_model_name,
    )
    version = model_info.registered_model_version
    if version is None:
        raise RuntimeError("MLflow did not return a registered model version")
    client = MlflowClient()
    client.set_model_version_tag(
        registered_model_name, version, "release_id", release_id
    )
    client.set_registered_model_alias(registered_model_name, "candidate", version
        )
    return str(version)
```

The registered name is `freshmart-demand-global`. MLflow assigns each immutable model version a numeric identifier such as 1, 2, or 3; the human-readable release ID is stored separately as a model-version tag, for example `release_id=a2-e10-001`. Aliases encode lifecycle (`candidate`, `champion`, `archived`) and are updated only after approval. A version includes model signature, feature list, source commit, image digest, data snapshot/hash, parameters, metrics, and model card. Registration is not promotion: E10 is a candidate because public proxy evidence cannot pass FreshMart business or uncertainty gates. MLflow supports model versions, aliases, lineage, and controlled transitions [6].

3.5 Validation results and analysis

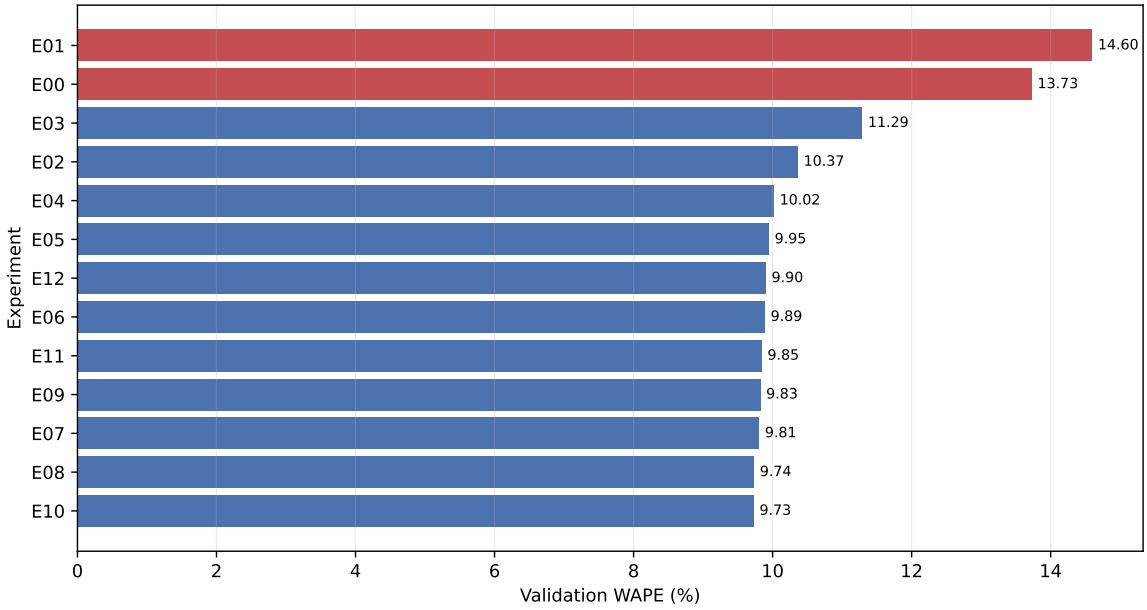


Figure 2: Validation WAPE for all 13 executed configurations. Red bars are rule baselines.

Table 4: Executed validation results. Cost is the declared asymmetric proxy per actual unit. Runtime combines local fit and validation inference; baseline times round to 0.00 seconds.

ID	Configuration	WAPE %	Bias %	RMSE	Cost proxy	Runtime s
E00	Seasonal naive 7d	13.73	0.99	12.18	0.2010	0.00
E01	Moving average 7d	14.60	0.44	13.06	0.2168	0.00
E02	Ridge, core	10.37	-0.69	9.14	0.1589	1.21
E03	HGB minimal 31	11.29	-1.54	9.99	0.1771	2.89
E04	HGB core 15	10.02	-0.03	8.86	0.1505	2.70
E05	HGB core 31	9.95	-0.18	8.80	0.1502	4.84
E06	HGB core 63	9.89	-0.32	8.75	0.1500	9.17
E07	HGB extended reg.	9.81	-0.56	8.68	0.1499	7.99
E08	HGB extended deep	9.74	-0.36	**8.60**	0.1478	21.05
E09	HGB Poisson	9.83	-0.27	8.70	0.1487	10.49
E10	HGB Poisson deep	**9.73**	-0.35	8.61	**0.1477**	20.54
E11	HGB conservative	9.85	-0.28	8.71	0.1491	16.68
E12	HGB extended + `log1p`	9.90	-1.25	8.80	0.1548	8.32

Five patterns matter. First, both rule baselines are weakest; E02 Ridge already reduces validation WAPE by 3.36 points, so much of the predictable signal is calendar/lag structure. Second, the minimal GBDT (E03) underperforms Ridge, while the core features cut WAPE to 10.02%: causal rolling and same-weekday summaries matter more than tree depth alone. Third, extended features and larger trees produce diminishing returns: E10 improves about 0.01 point over E08 after rounding. The difference is not evidence of statistical superiority without multiple rolling folds. Fourth, the clean loss comparison is E07 versus E09; Poisson is 0.02 point worse, so these results do not support a general claim that Poisson is better. E08 versus E10 is confounded by iterations and L2. Fifth, E12's `log1p` target worsens WAPE from 9.81% to 9.90% and increases negative bias from −0.56% to −1.25% versus otherwise-identical E07; raw-unit preprocessing is therefore retained.

E10 passes the predeclared prototype rule: lowest validation WAPE, absolute bias below 3%, and more than two percentage points better than E00. It is recommended as the *candidate* because the deterministic primary-metric rule selects it and its runtime remains negligible relative to the batch window, not because this experiment proves Poisson intrinsically superior. E08 remains an equally credible operational challenger: it has the best RMSE, nearly identical WAPE, and the E08/E10 comparison is not clean. Production selection should aggregate several rolling-origin folds and estimate uncertainty before preferring either.

3.6 Untouched test-quarter evidence

After selection, E10 was refit on train plus validation and scored once on 2017 Q4. Table 5 reports the stored metrics rounded to two decimals (four decimals for the cost proxy).

Table 5: Untouched test comparison (46,000 store–item–days).

Model	WAPE %	Bias %	RMSE	sMAPE %	Fill proxy %	Cost proxy
Seasonal naive (E00)	16.60	2.44	12.05	18.56	92.92	0.2368
Selected candidate (E10)	**10.94**	**-0.23**	**7.76**	**12.57**	**94.41**	**0.1653**

The absolute WAPE reduction is 5.66 points and relative reduction is 34.1%. The cost proxy falls 30.2%. However, test WAPE is 1.21 points higher than validation WAPE. This is consistent with a seasonal shift, although that explanation is not tested here, and reinforces the need for rolling folds. Figure 3 illustrates one series; operational acceptance must be based on aggregate and slice metrics, not a favorable trace.

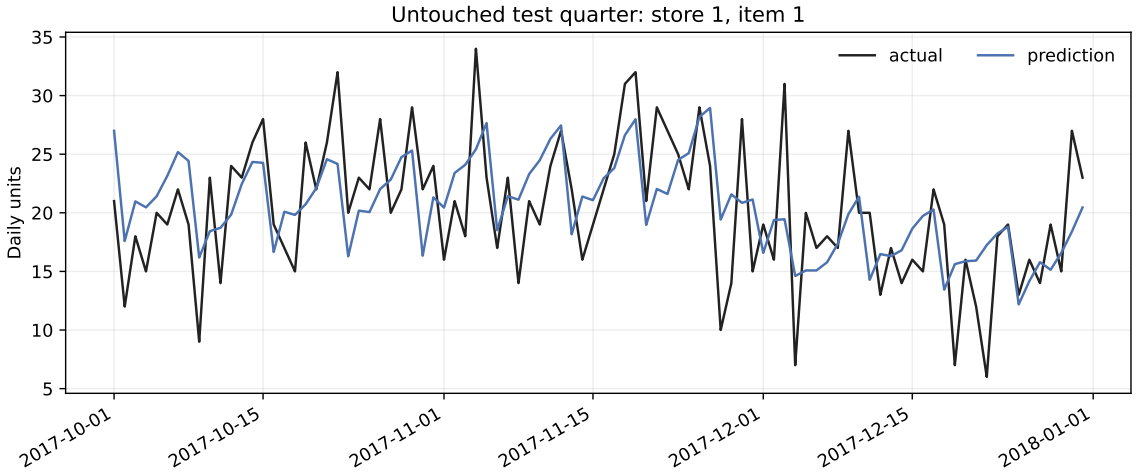


Figure 3: E10 predictions for store 1, item 1 in the untouched test quarter.

4 Workflow Orchestration Design (20%)

4.1 DAG architecture and dependencies

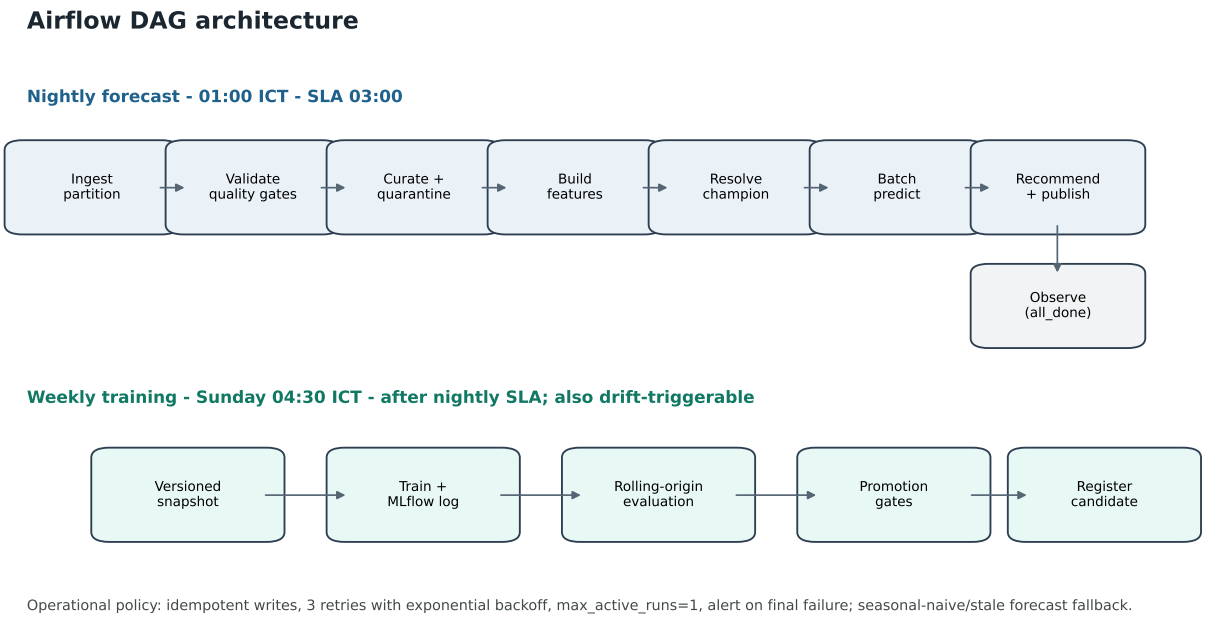


Figure 4: Two Airflow DAGs and their explicit dependencies.

The nightly DAG is scheduled at 01:00 Asia/Ho_Chi_Minh after source close and must publish before 03:00. Its `start_date` is a timezone-aware `pendulum.datetime(..., tz="Asia/Ho_Chi_Minh")`; therefore the cron is interpreted as 01:00 ICT, not UTC. It ingests a date partition, validates it, curates data, materializes a point-in-time feature snapshot, resolves an immutable champion version, predicts, applies replenishment constraints, and atomically publishes. Observability runs with `all_done` so failures still emit an event. The companion file contains both executable DAG objects. The weekly training DAG has two trigger types: scheduled cron at Sunday 04:30 ICT, after the nightly 03:00 SLA, and a manual/API DagRun with `trigger_type=approved_drift` after a sustained drift alert receives human approval. It versions a snapshot, trains/logs, evaluates rolling origins, applies promotion gates, and registers a candidate. Canary/shadow approval changes the registry alias; the training DAG never deploys directly. These explicit DAG/task dependencies follow Airflow's orchestration model [7].

Table 6: Task dependency, output, and recovery behavior.

Task	**Depends on**	**Durable output**	**Failure/recovery**
Ingest	Source availability	Raw date partition + manifest	Retry network errors; idempotent key; alert late source
Validate	Ingest	Quality report/quarantine list	Fail closed; do not consume invalid partition
Curate/features	Validation	Versioned tables/snapshot	Atomic staging; rerun same date safely
Resolve champion	Registry	Name + immutable version	Use last approved version; never "latest"
Predict	Features + champion	Partitioned predictions	Retry failed partitions; fallback if incomplete
Recommend/publish	Predictions + ERP state	Forecast-store transaction + ERP file	Validate constraints; atomic swap; stale/baseline fallback
Observe	All upstream states	Metrics, lineage, alert event	`all_done`; notification itself has a secondary route
Train/track	Versioned snapshot	MLflow run + model artifact	Weekly/on demand; failures never affect current champion
Evaluate/register	Completed run	Evaluation report + candidate version	Gate failure ends normally without promotion

4.2 Airflow definition and operational controls

The complete import-safe DAG is included in `dags/demand_forecasting_dag.py`. It uses `max_active_runs=1` to prevent concurrent writes, three retries with exponential backoff for transient failures, a two-hour execution timeout, a final failure callback, and explicit task returns rather than hidden cross-task state. The validation task reads a measured `quality_report.json` written beside the landed partition and fails on either completeness below 98% or any failed check; no pass value is hard-coded.

Listing 3: Complete nightly DAG header with retry policy, timezone, schedule, and concurrency controls.

```
DEFAULT_ARGS = {
    "owner": "ml-platform",
    "depends_on_past": False,
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=30),
    "execution_timeout": timedelta(hours=2),
    "on_failure_callback": notify_failure,
}

TIMEZONE = "Asia/Ho_Chi_Minh"


if AIRFLOW_AVAILABLE:

    @dag(
        dag_id="freshmart_nightly_forecast",
        schedule="0 1 * * *",
        start_date=pendulum.datetime(2026, 1, 1, tz=TIMEZONE),
        catchup=False,
        max_active_runs=1,
        default_args=DEFAULT_ARGS,
        tags=["freshmart", "inference", "batch"],
    )
    def nightly_forecast() -> None:
        """Create and publish forecasts before the 03:00 business SLA."""

        # Task definitions follow; the explicit wiring is at the end of the DAG.
```

The dependency wiring is equally explicit, including the monitoring task's `all_done` trigger:

Listing 4: Monitoring trigger and TaskFlow dependency wiring.

```
        @task(trigger_rule="all_done")
        def observe(publication: dict[str, str]) -> None:
            """Emit SLA, volume, feature-drift, and prediction-distribution
               metrics."""

            LOGGER.info("observe publication=%s", publication)

        raw = ingest("{{ ds }}")
        checked = validate(raw)
        curated = curate(checked)
        features = build_point_in_time_features(curated)
        model = resolve_champion(features)
        predictions = batch_predict(features, model)
        orders = recommend_orders(predictions)
        published = publish(orders)
        observe(published)
```

The training workflow and its trigger type are also explicit; task definitions between the header and wiring are omitted only from this compact excerpt:

Listing 5: Weekly training schedule, reviewed-drift trigger, and dependency wiring.

```python
@dag(
    dag_id="freshmart_weekly_training",
    schedule="30 4 * * 0",
    start_date=pendulum.datetime(2026, 1, 1, tz=TIMEZONE),
    catchup=False,
    max_active_runs=1,
    default_args=DEFAULT_ARGS,
)
def weekly_training() -> None:
    # Task definitions: validate snapshot, train/track, evaluate, register.
    trigger = "{{ dag_run.conf.get('trigger_type', 'scheduled') }}"
    snapshot = validate_training_snapshot(trigger)
    run = train_and_track(snapshot)
    evaluation = rolling_origin_evaluation(run)
    register_candidate(run, evaluation)
```

Operational monitoring has four layers: (1) task/SLA metrics such as duration, retries, and on-time publication; (2) data quality and freshness; (3) feature/prediction drift and delayed WAPE/bias/coverage; and (4) business waste, OSA, margin, and override FVA. Alerts are severity-routed: paging for missed 03:00 SLA, corrupt production data, or publish failure; ticket for slow drift; dashboard only for informational slice changes. Secrets are references resolved from Vault/secret manager, never literals in DAG source.

5 Code Quality and Documentation (20%)

5.1 Standards

The implementation follows PEP 8 naming, small single-purpose modules, deterministic functions, and a `main()` boundary. Public functions have docstrings and typed signatures; configuration and paths are passed rather than embedded in model logic. Comments explain *why*, for example why target statistics are shifted and why Airflow is optional locally. Errors are specific and fail early (hash mismatch, unexpected schema, duplicate composite keys, negative demand, empty/overlapping partitions, or missing promotion candidate).

Seven deterministic unit tests verify perfect/directional metrics, negative-prediction clipping, causal lags, strict temporal ordering, environment expansion, and frame-independent trend anchoring. The complete source compiles with `py_compile`; all seven tests pass. The most important leakage and train/serve controls are visible in the feature code:

Listing 6: Complete causal feature function, including all shifted rolling features and the fixed trend anchor.

```python
def engineer_features(
    frame: pd.DataFrame, *, trend_anchor_date: str = "2013-01-01"
) -> pd.DataFrame:
    """Create causal features anchored identically in training and inference."""

    data = frame.copy()
    groups = data.groupby(["store", "item"], observed=True)["sales"]
    for lag in (1, 7, 14, 21, 28):
        data[f"lag_{lag}"] = groups.shift(lag).astype("float32")

    shifted = groups.shift(1)
    shifted_group = shifted.groupby([data["store"], data["item"]], observed=True)
    data["rolling_mean_7"] = (
        shifted_group.rolling(7).mean().reset_index(level=[0, 1], drop=True).
            astype("float32")
    )
    data["rolling_std_7"] = (
        shifted_group.rolling(7).std().reset_index(level=[0, 1], drop=True).
            astype("float32")
    )
    data["rolling_mean_28"] = (
        shifted_group.rolling(28).mean().reset_index(level=[0, 1], drop=True).
            astype("float32")
    )
    data["same_weekday_mean_4"] = data[["lag_7", "lag_14", "lag_21", "lag_28"]].
        mean(axis=1)

    data["day_of_week"] = data["date"].dt.dayofweek.astype("int8")
    data["month"] = data["date"].dt.month.astype("int8")
    data["day_of_year"] = data["date"].dt.dayofyear.astype("int16")
    data["is_weekend"] = (data["day_of_week"] >= 5).astype("int8")
    anchor = pd.Timestamp(trend_anchor_date)
    data["trend_days"] = (data["date"] - anchor).dt.days.astype("int16")
    data["dow_sin"] = np.sin(2 * np.pi * data["day_of_week"] / 7).astype("float32
        ")
    data["dow_cos"] = np.cos(2 * np.pi * data["day_of_week"] / 7).astype("float32
        ")
    data["doy_sin"] = np.sin(2 * np.pi * data["day_of_year"] / 365.25).astype("
        float32")
    data["doy_cos"] = np.cos(2 * np.pi * data["day_of_year"] / 365.25).astype("
        float32")

    return data.dropna().reset_index(drop=True)
```

5.2 Configuration and environment variables

Experiment definitions are data, not branching code. `config/experiments.yaml` declares seed, checksum, fixed trend anchor, split dates, 3% bias gate, and all 13 runs. `config/pipeline.yaml` declares the operational endpoints and thresholds. Strings such as `${VAR:-default}` are ordinary YAML scalars; the submitted loader explicitly expands them after `yaml.safe_load`. `.env.example` documents required names but contains no secrets. This separation allows dev/staging/prod to share code while using distinct storage, registry, and forecast tables.

Listing 7: Operational configuration with explicit gates and fallbacks.

```yaml
environment: ${ENVIRONMENT:-development}
timezone: Asia/Ho_Chi_Minh
raw_data_uri: ${RAW_DATA_URI:-s3://freshmart-ml/raw}
curated_data_uri: ${CURATED_DATA_URI:-s3://freshmart-ml/curated}
forecast_table: ${FORECAST_TABLE:-freshmart.forecasts}
```

```yaml
mlflow_tracking_uri: ${MLFLOW_TRACKING_URI:-http://mlflow:5000}
model_name: freshmart-demand-global
quality_gates:
  minimum_store_completeness: 0.98
  maximum_duplicate_fraction: 0.0001
  maximum_negative_sales_fraction: 0.0
promotion_gates:
  maximum_wape: 0.27
  maximum_absolute_bias: 0.03
  p90_coverage_minimum: 0.88
  p90_coverage_maximum: 0.92
fallback:
  stale_forecast_max_age_hours: 30
  baseline: seasonal_naive_7d
```

Listing 8: Complete environment expansion and configuration-loading functions.

```python
ENV_PATTERN = re.compile(r"\$\{([A-Za-z_][A-Za-z0-9_]*)(?::-([^}]*))?\}")


def _expand_env(value: Any) -> Any:
    """Recursively expand ``${VAR}`` and ``${VAR:-default}`` in parsed YAML."""

    if isinstance(value, dict):
        return {key: _expand_env(item) for key, item in value.items()}
    if isinstance(value, list):
        return [_expand_env(item) for item in value]
    if not isinstance(value, str):
        return value

    def replace(match: re.Match[str]) -> str:
        name, default = match.group(1), match.group(2)
        if name in os.environ:
            return os.environ[name]
        if default is not None:
            return default
        raise ValueError(f"Required environment variable is not set: {name}")

    return ENV_PATTERN.sub(replace, value)


def load_config(path: str | Path) -> dict[str, Any]:
    """Parse a YAML file, then expand deployment-specific environment values."""

    loaded = yaml.safe_load(Path(path).read_text(encoding="utf-8"))
    if not isinstance(loaded, dict):
        raise ValueError("Top-level configuration must be a mapping")
    return _expand_env(loaded)
```

The companion `DDM501_Assignment2_25MS23263_TranVietHoa_submission.zip` includes source, configuration, tests, data, metrics, predictions, candidate model, manifest, figures, LaTeX, and final PDF. Its README gives exact run/test commands, data provenance/checksum, artifact locations, and the warning that the dataset is a proxy. A production CI pipeline would run format/lint/type checks, unit and data-contract tests, a small deterministic integration sample, DAG import tests, image scanning, and a reproducibility smoke run before publishing an immutable image.

6 Reproducibility and Versioning Strategy (10%)

6.1 Code, data, configuration, and model versions

Code. Trunk-based development uses short-lived `feature/*` branches, protected pull requests, and required CI. Releases are annotated semantic tags such as `v1.2.0`; urgent production changes use

`hotfix/*` and merge back to trunk. The Git commit SHA is attached to each MLflow run and container label.

Data. Immutable raw partitions are addressed by date and checksum. Curated snapshots have a manifest containing source partitions, dbt/feature version, row counts, quality results, and event-time cut-off. DVC stages in `dvc.yaml` connect the raw file, configuration, source, results, and candidate manifest; production would store large objects in versioned S3/GCS rather than Git [8].

Model. Registry name, immutable model version, training-data snapshot, feature-contract version, source/image digests, signature, metrics, approval, and lifecycle alias are stored together. Predictions contain `model_version`, `feature_version`, and `generated_at`, permitting line-level audit and rapid rollback.

The concrete code/data release workflow is:

Listing 9: Code and data versioning commands.

```
git switch -c feature/demand-pipeline
git commit -am "Add reproducible demand experiment"
# Open, review, and merge the pull request into trunk:
gh pr create --fill
gh pr merge --squash --delete-branch
git switch main
git pull --ff-only
git tag -a v1.2.0 -m "Validated candidate release"
dvc repro
dvc push
```

The experiment runner stores `git rev-parse HEAD` as the `git_commit` MLflow tag; when executed outside Git it records the explicit value `unversioned`, rather than inventing lineage.

6.2 Determinism and containerization

The random seed 501 is loaded from the versioned configuration, set in NumPy, and passed into model construction:

Listing 10: Run-level deterministic seed initialization.

```
def main() -> None:
    args = parse_args()
    config_path = ROOT / args.config
    config = load_config(config_path)
    seed = int(config["project"]["random_seed"])
    np.random.seed(seed)
```

Listing 11: The same seed is passed to the stochastic estimator.

```
    if config["model"] == "hist_gradient_boosting":
        return HistGradientBoostingRegressor(
            learning_rate=float(config["learning_rate"]),
            max_iter=int(config["max_iter"]),
            max_leaf_nodes=int(config["max_leaf_nodes"]),
            min_samples_leaf=int(config["min_samples_leaf"]),
            l2_regularization=float(config["l2_regularization"]),
            loss=str(config["loss"]),
            random_state=random_seed,
            early_stopping=False,
        )
```

Split boundaries and the experiment matrix are committed. Dependencies are pinned; the saved manifest records the exact feature order and checksum. Exact bitwise equality is not promised across CPU architectures or library builds, so acceptance is based on a small metric tolerance and matching lineage. The Dockerfile runs as a non-root user, sets a deterministic entrypoint, and packages the same code used locally:

Listing 12: Submitted container definition.

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    MPLCONFIGDIR=/tmp/matplotlib

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

RUN useradd --create-home --uid 10001 appuser && chown -R appuser:appuser /app
USER appuser

ENTRYPOINT ["python", "scripts/run_experiments.py"]
CMD ["--config", "config/experiments.yaml"]
```

The minimal reproduction is:

```
python3 scripts/run_experiments.py --config config/experiments.yaml
python3 -m unittest discover -s tests -v
```

It produces CSV/JSON metrics, plots, test predictions, a serialized candidate, and its manifest. To mirror tracking, install the optional production requirements and add `-mlflow`. This report does not claim that the heavy Airflow/MLflow services were needed to run the local statistical experiment; their executable definitions and integration points are supplied for the production workflow.

7 Limitations, Risks, and Next Decision

1. **External validity.** The public data has only date, store, item, and sales. It cannot validate FreshMart effects from promotions, weather, prices, inventory, Tet, shelf life, new items, or stockout censoring.
2. **Task scope.** The prototype is one-day point forecasting. Production requires horizons 1–3 and calibrated P10/P50/P90 outputs. Quantile LightGBM plus split-conformal calibration must be evaluated separately [9].
3. **Selection uncertainty.** E10 and E08 differ by about 0.01 WAPE point after rounding on one validation quarter, while their clean causal difference is not isolated. Multiple rolling origins, paired series-level errors, and uncertainty intervals are required before asserting superiority.
4. **Business causality.** A forecast does not prove waste or availability improvement. The fill and cost values are proxies. The 40-treatment/40-control, eight-week difference-in-differences pilot from Assignment 1 remains the business gate.
5. **Operational realism.** The local run validates code and model behavior, not production throughput, Airflow scheduling, cloud IAM, API latency, or disaster recovery. Load, security, canary, restore, and failover tests are required.

The next decision is therefore to keep the current process as champion, register E10 as candidate, and run it in shadow mode on a versioned FreshMart snapshot. It may progress to canary only if it beats both production baselines across rolling folds, meets WAPE/bias/coverage and segment gates, completes the nightly SLA, and passes the data-contract and rollback drills.

8 Conclusion

Assignment 2 turns the earlier high-level design into an auditable MLOps workflow. It provides a stage-by-stage architecture, explicit quality and promotion gates, 12 initially declared configura-

tions plus a post-review preprocessing ablation, stored run artifacts, an untouched-quarter test, two timezone-correct Airflow DAGs, typed and tested code, executable configuration loading, DVC/Git lineage, MLflow model registration, and a non-root container. E10 materially beats the seasonal-naive proxy baseline, but the strongest conclusion is appropriately narrower: the implementation is ready to become a FreshMart shadow candidate, not a production champion. That distinction connects model evidence to operational reliability and measured business value.

A Rubric-to-Artifact Traceability

Requirement	Evidence in report	Executable artifact
Pipeline diagram, stages, rationale	Figure 1, Table 1, Section 2	`src/data.py`, `src/experiment.py`
Minimum 8–10 configurations	12 initially declared runs plus E12 ablation, preprocessing pair, and full results, Section 3	`config/experiments.yaml`, `artifacts/results.csv`
MLflow logging/registration	Tracking and registry subsection	`src/tracking.py`, candidate manifest
Results and recommendation	Figures 2–3, Tables 5–6	test metrics/comparison JSON and predictions CSV
DAG, tasks, dependencies, schedule	Figure 4, Section 4	`dags/demand_forecasting_dag.py`
Failure, alerting, monitoring	Sections 2.3 and 4.2	callbacks, retries, `all_done`, fallback config
Standards, docstrings, hints, comments	Section 5 and code excerpts	`src/`, seven unit tests
Config and environment variables	Section 5.2	YAML files, `src/config.py`, and `.env.example`
Code/data/model versioning	Section 6	Git/DVC commands, checksums, registry manifest
Seeds and Dockerfile	Section 6.2	seed 501 snippets; `Dockerfile`

References

[1] T. V. Hoa, *ML System Design Document: Perishable-Food Demand Forecasting and Replenishment Assistant*, DDM501 Individual Assignment 1, 2026.

[2] Kaggle, "Store Item Demand Forecasting Challenge," training data, 2018. https://www.kaggle.com/competitions/demand-forecasting-kernels-only/data

[3] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, 38(4), pp. 1346–1364, 2022.

[5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, 2021. https://otexts.com/fpp3/

[6] MLflow Project, "Model Registry," documentation. https://mlflow.org/docs/latest/ml/model-registry/

[7] Apache Airflow, "DAGs," documentation. https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html

[8] Iterative, "DVC: Data Version Control," documentation. https://dvc.org/doc

[9] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," arXiv:2107.07511, 2021.

[10] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems*, vol. 28, 2015.